

Aprendizaje Computacional

Dr. José Francisco Martínez Trinidad

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso	2
2. Aprendizaje	4
3. Clasificadores lineales y perceptrón	7
4. Convergencia y generalización (perceptrón)	10
5. Máquinas de vectores de soporte (SVM)	14
6. SVM revisitada: forma dual y kernels	18
7. Preparación de datos y redes neuronales	24
8. Neurona artificial y separabilidad lineal	27
9. Descenso de gradiente y backpropagation	32
10.Redes convolucionales	39
11.Redes residuales y autoencoders	44

1. Introducción al Curso

25 de agosto de 2026

Resumen rápido. Sesión inicial: contenido y dinámica del curso, temario, criterios de evaluación y el calendario del proyecto. La segunda mitad fue la introducción propiamente dicha: qué es aprender, por qué generalizar no es memorizar, de qué disciplinas se alimenta el área y la formalización de la función f que se quiere aprender contra la hipótesis h .

Info general

Contacto:

- Dr. José Francisco Martínez Trinidad
- Correo: fmartine@inaoep.mx

Temario:

1. Introducción y clasificadores lineales
2. Perceptrón
3. SVM
4. Clasificadores basados en distancia
5. ANNs
6. Evaluación, validación y overfitting
7. Métodos bayesianos
8. Árboles de decisión
9. Reglas de clasificación
10. Reglas de asociación
11. Selección de atributos
12. Clustering (K-means, gaussian mixture, jerárquicos)

Idea clave

No se trata de usar las herramientas como una caja negra. El objetivo es, como investigadores, saber cómo usarlas para generar nuevo conocimiento, y por eso hay que conocerlas a detalle: para poder innovar y proponer alternativas.

Evaluación

80 % proyecto:

- Equipo de dos personas
- Seleccionar un artículo reciente
- Proponer una solución alternativa
- Tres presentaciones

- Reporte final

20 % examen oral, a la entrega del proyecto, sobre un tema aleatorio del temario.

Calendario

- **25 de agosto.** Buscar un artículo publicado en 2026 que sea de interés, en una revista JCR.
- **3 de septiembre.** Entregar por correo una lista que indique cuál artículo seleccionó cada estudiante o equipo.
- **29 de septiembre o 1 de octubre.** Primera presentación:
 - Descripción del problema abordado en el artículo
 - Breve descripción del trabajo relacionado que menciona el artículo
 - Breve descripción de la solución propuesta en el artículo
 - Descripción de los experimentos que soportan dicha solución, incluyendo tablas y gráficas tomadas del artículo
 - Conclusiones a las que llegan los autores
 - Análisis de debilidades y posibles modificaciones para mejorar la propuesta de los autores
- **27 o 29 de octubre.** Segunda presentación: resumen del problema abordado.
- **24 o 26 de noviembre.** Presentación final: resumen del problema abordado y de la solución final.
- **30 de noviembre.** Entrega del reporte final. Se entrega en forma de artículo, con formato de <https://www.springer.com/>.

El reporte debe tener más o menos 12 páginas e incluir título, resumen, introducción, trabajo relacionado, solución propuesta, experimentos, conclusiones y trabajo futuro, y referencias.

Revistas JCR

- Expert Systems with Applications
- Knowledge-Based Systems
- Journal of Machine Learning Research

Buscador: <https://www.webofscience.com/wos/woscc/basic-research>

Recursos

- Weka 3
- scikit-learn

Ojito...

Hay que tratar de contactar al autor del artículo para que nos dé sus recursos y poder compararnos.

2. Aprendizaje

Que es el aprendizaje?

Es una disciplina dentro de las ciencias de la computación que surge como motivación de hacer que las computadoras realicen tareas complejas como las que hacen los humanos.

Aprender significa adquirir conocimiento, entendimiento o una habilidad de manera empírica o por estudio. La pregunta que queda abierta es cómo determinamos el éxito o el fracaso de la tarea.

Se busca que se logre un proceso de aprendizaje para que el sistema sea capaz de generalizar.

Idea clave

Generalizar no es memorizar. Aprender permite responder a problemas nuevos con el conocimiento adquirido.

Al inicio hubo varias tendencias. Una de ellas fue ver cómo aprenden los animales y de ahí trasladarlo a la máquina, con la premisa de que quizá el aprendizaje en un animal es diferente al de un ser humano, pero no resultó cierto.

Ojito...

No hay una demostración universal de lo que es el aprendizaje. Hay muchas definiciones y muchas escuelas.

Tipos de aprendizaje

- Supervisado
- No supervisado
- Por refuerzo

Tareas relacionadas con el aprendizaje

- Reconocimiento
- Diagnóstico
- Planeación
- Control de robots
- Predicción

Aprender puede ser percibir cambios en el comportamiento gracias a la experiencia. En nuestro contexto serían mejoras en actividades ya conocidas, y aprender tareas nuevas.

Hacer que las máquinas aprendan es muy útil, ya que nos simplifica la vida en muchos contextos: nos ayuda a hacer tareas complejas más rápido y a analizar datos masivos de una manera rápida.

Disciplinas relacionadas

- Estadística
- Lógica

- Matemáticas discretas
- Y muchas más

Todas las teorías que hay sobre cómo funciona el cerebro han ayudado bastante, ya que se trata de emular lo que se cree que ocurre en el cerebro humano cuando ocurre el aprendizaje, y trasladarlo a la computadora.

Otra disciplina que se vincula mucho con el ML es el control, la teoría de control. Igual los modelos psicológicos y los modelos evolutivos, entre muchos más.

Qué es lo que se aprende?

Diferentes estructuras computacionales: funciones, programas lógicos, máquinas de estado finito, gramáticas y sistemas para la resolución de problemas.

La idea es que tengo mi problema y tengo algunos ejemplos de él. Le doy a la computadora los ejemplos y la computadora me da como resultado el programa que me ayuda a generalizar cómo son esos ejemplos, de tal manera que cuando llegue un dato nuevo pueda clasificarlo, identificarlo y reconocerlo correctamente.

Más formalmente, se tiene una función f , que es la que queremos aprender, y otra función h , la hipótesis. Ambas reciben un vector de valores con N componentes. Queremos encontrar la función h cuya salida sea $h(X)$, y que sea capaz de reconocer un dato nuevo.

Supervisado y no supervisado

Hay dos tipos de aprendizaje, bueno tres, pero estos son los más comunes.

- **Supervisado.** Los datos de entrenamiento están etiquetados. A la función h , que es la que voy a aprender, le voy a dar un dato x , y esa función tiene que ser capaz de reconocer la nueva entrada.
- **No supervisado.** Los datos de entrenamiento no están etiquetados, y el algoritmo encuentra la relación entre los datos.

Vector de entrada

Al vector de entrada se le llama patrón, feature vector, instancia o ejemplo. Sus componentes son las variables o características, y esas variables pueden tomar valores reales, discretos o categóricos.

La salida puede ser un valor real, y entonces se trata de un predictor, o discreto, y entonces se trata de un clasificador.

Modos de entrenamiento

Hay varios modos de entrenamiento, dependiendo de los datos:

- Por batch
- De manera incremental
- En línea

Evaluación

Cuando ya tengo mi función h , ¿cómo la evalúo? ¿Cómo sé que está bien? Para eso se necesita un conjunto de prueba, que nos ayuda a ver qué tan bien la función h realiza su tarea, midiendo el error de h sobre ese conjunto. El conjunto de prueba es una partición del dataset original.

3. Clasificadores lineales y perceptrón

27 de agosto de 2026

Resumen rápido. Retomamos la idea de aprender una función que generalice y la aterrizamos en un ejemplo de control de acceso por rostro. De ahí salieron los clasificadores lineales, las funciones de pérdida y el perceptrón como algoritmo para ajustar los parámetros.

Recap

ML se trata de encontrar una función que fitee unos datos de entrenamiento para que sea capaz de generalizar. El proceso es:

1. Suponemos una función f que modela lo que queremos
2. Construimos hipótesis
3. Queremos construir la función h que emule a f
4. Entrenamos a h para que sea capaz de generalizar
5. Testeamos

Ejemplo: control de acceso por rostro

Supongamos que queremos construir un sistema que permita pasar a personas cuyos rostros sean conocidos para entrar al instituto. Tenemos los datos de las caras, y una imagen es un arreglo de bits.

Tenemos que construir una función

$$f : \mathbb{R}^d \rightarrow \{-1, 1\}$$

Es discreta porque es una tarea de clasificación. El clasificador tendrá un conjunto de vectores de entrenamiento con etiquetas, y estos vectores nos ayudarán a entrenar la función. Entonces queremos construir la función solo con los datos de entrenamiento, o sea con nuestra experiencia empírica.

Una manera sería ver un píxel de la imagen, de tal forma que al estar presente acepta a esa persona y la rechaza en otro caso, pero esto tiene varias limitaciones. ¿La función es útil para otras imágenes que no estén en el dataset? ¿Es capaz de generalizar?

Idea clave

Lo que se trata es de encontrar un modelo, una función que nos resuelva esa tarea de aprendizaje. No queremos funciones que fiteen perfectamente al conjunto de datos, ni súper generales, queremos algo en medio.

Selección de modelo

Y cómo se encuentran clasificadores que generalicen bien. La clave es *restringir* el conjunto de funciones binarias que uno se permite considerar. Se busca una clase de funciones tal que, si una función de la clase funciona bien en el entrenamiento, es probable que también funcione bien en las imágenes que no se han visto.

Idea clave

El problema de selección de modelo. La clase correcta no puede ser demasiado grande, porque entonces contiene demasiadas funciones claramente distintas y una de ellas va a memorizar el entrenamiento por casualidad, como la del píxel único. Pero tampoco demasiado chica, porque entonces se corre el riesgo de no encontrar ninguna función que funcione bien ni siquiera en el entrenamiento. Encontrar esa clase es uno de los problemas centrales del aprendizaje automático.

Fijar la clase de funciones en los clasificadores lineales, que es lo que sigue, es exactamente tomar esa decisión.

Clasificadores lineales

Lo que hacemos es definir una función lineal:

$$f(x; \theta) = \text{sign}(\theta_1 x_1 + \dots + \theta_d x_d) = \text{sign}(\theta^\top x)$$

Se toma el signo para nuestro problema porque solo estamos considerando el caso binario: acepta o rechaza la entrada al edificio.

El modelo encontrará una línea, un plano que permita separar a las personas que pueden o no entrar al edificio. Ese plano es justo el lugar donde $\theta^\top x = 0$, y θ es el vector normal a él, así que ajustar θ es girar y mover el plano.

El chiste es encontrar la función a partir de las entradas que tengo en el entrenamiento. Cuando no es posible construir la función, queremos entonces descubrir otra que cometa el menor error posible.

Ojito...

En nuestro problema el clasificador lineal no guarda info de los píxeles vecinos, solo evaluaría el píxel individual.

Funciones de pérdida

Mencionamos que queremos encontrar el error. Esto se mide con la diferencia del dato esperado con el dato obtenido, y hay diferentes medidas de error.

A las funciones de error se les llama **loss**, función de error o función de pérdida, y hacen lo que mencionábamos: dicen qué tanta diferencia hay entre el dato observado y el esperado. Son esas funciones las que se tratan de optimizar.

El perceptrón

Queremos encontrar un clasificador lineal que minimice el error, o sea encontrar θ que minimice el error de entrenamiento:

$$E(\theta) = \frac{1}{n} \sum_{i=1}^n [1 - \delta(y^{(i)}, f(x^{(i)}; \theta))]$$

Donde δ vale 1 si sus dos argumentos coinciden y 0 si no, así que $1 - \delta$ vale 1 exactamente cuando el ejemplo quedó mal clasificado. Dicho de otra forma, $E(\theta)$ es la fracción de ejemplos de entrenamiento en los que el modelo se equivoca.

¿Pero cómo construir ahora esa función lineal? ¿Cómo ajustar los parámetros? ¿Hay alguna manera razonable de setear los parámetros de θ ?

Una opción es ajustar incrementalmente cada parámetro para que veamos que está bien. Un algoritmo simple de este tipo es el **perceptron update rule**:

$$\text{si } y^{(i)}(\theta^\top x^{(i)}) \leq 0 \quad \text{entonces} \quad \theta' = \theta + y^{(i)}x^{(i)}$$

En otras palabras, los parámetros son actualizados solo si hay error, y estos updates tienden a corregir errores. En nuestro caso, la clasificación de caras, cuando el modelo hace un error el signo no coincide con el de la etiqueta.

Supongamos que hacemos un error, y que los parámetros actualizados están dados por θ' . Si consideramos clasificar la misma imagen después del update:

$$\begin{aligned} y^{(i)}(\theta'^\top x^{(i)}) &= y^{(i)}((\theta + y^{(i)}x^{(i)})^\top x^{(i)}) \\ &= y^{(i)}(\theta^\top x^{(i)}) + (y^{(i)})^2 \|x^{(i)}\|^2 \\ &= y^{(i)}(\theta^\top x^{(i)}) + \|x^{(i)}\|^2 \end{aligned}$$

Como $y^{(i)} \in \{-1, 1\}$ su cuadrado siempre es 1, y $\|x^{(i)}\|^2 > 0$. O sea que el update le suma algo positivo, empujando ese ejemplo hacia el lado correcto del plano. Eso es lo que quiere decir que los updates tienden a corregir errores.

Ojito...

Esto da origen al reconocedor del perceptrón, y este reconocedor es útil para problemas lineales: solo puede separar con una línea o un plano, pero jamás podrá separar datos que no sean linealmente separables.

4. Convergencia y generalización (perceptrón)

1 de septiembre de 2026

Resumen rápido. En esta sesión vimos la convergencia del perceptrón, y la definición formal, que era el margen geométrico y cerramos con la intro a SVM

Punto de partida y supuestos

Seguimos con clasificadores lineales que pasan por el origen:

$$f(x; \theta) = \text{sign}(\theta^\top x),$$

donde θ denota los parámetros que tenemos que estimar en base a los ejemplos de entrenamiento, con ejemplos x y etiquetas y

Sea k el número de parámetros actualizados y $\theta^{(k)}$ el vector de parámetros después de k actualizaciones. Cada vez que el modelo se equivoca hace un update con la regla del perceptrón, y ese update es un k más

Para poder demostrar algo hacen falta dos supuestos:

1. **Las normas están acotadas:** existe R tal que $\|x^{(t)}\| \leq R$ para todo ejemplo.
2. **El clasificador lineal existe:** hay un θ^* , el θ ideal, que separa los datos con margen. Formalmente,

$$y^{(t)}(\theta^*)^\top x^{(t)} \geq \gamma > 0 \quad \text{para todo } t,$$

donde γ es el margen del clasificador, que es lo que nos dice que lo hace bien

Ojito...

En el ejemplo de las imágenes de las caras reconocidas para entrar al edificio, el segundo supuesto es justamente suponer que ese clasificador existe. Si los datos no son linealmente separables, no hay θ^* y nada de lo que sigue aplica

Convergencia en un número finito de updates

La convergencia se demuestra combinando dos resultados. El primero dice que el producto interno con el ideal crece linealmente, y el segundo que la norma cuadrada crece a lo más linealmente. Tomamos $\theta^{(0)} = 0$.

Parte 1: el producto interno crece linealmente

Supongamos que el update k se hizo sobre el ejemplo $(x^{(t)}, y^{(t)})$, o sea $\theta^{(k)} = \theta^{(k-1)} + y^{(t)} x^{(t)}$. Entonces

$$\begin{aligned} (\theta^*)^\top \theta^{(k)} &= (\theta^*)^\top \theta^{(k-1)} + y^{(t)} (\theta^*)^\top x^{(t)} \\ &\geq (\theta^*)^\top \theta^{(k-1)} + \gamma, \end{aligned}$$

donde la desigualdad es el supuesto del margen. Cada update le suma al menos γ , así que aplicándolo k veces desde $\theta^{(0)} = 0$:

$$(\theta^*)^\top \theta^{(k)} \geq k\gamma.$$

Parte 2: la norma cuadrada crece a lo más linealmente

Con el mismo update,

$$\begin{aligned}\|\theta^{(k)}\|^2 &= \|\theta^{(k-1)} + y^{(t)} x^{(t)}\|^2 \\ &= \|\theta^{(k-1)}\|^2 + 2 y^{(t)} (\theta^{(k-1)})^\top x^{(t)} + (y^{(t)})^2 \|x^{(t)}\|^2 \\ &\leq \|\theta^{(k-1)}\|^2 + R^2.\end{aligned}$$

El término de en medio se va porque el update ocurre precisamente cuando el ejemplo estaba mal clasificado, o sea cuando $y^{(t)} (\theta^{(k-1)})^\top x^{(t)} \leq 0$. Y el último se acota porque $(y^{(t)})^2 = 1$ y $\|x^{(t)}\| \leq R$. Repitiendo k veces:

$$\|\theta^{(k)}\|^2 \leq k R^2$$

Combinación: qué pasa con el ángulo

Si combinamos esos dos resultados podemos ver el coseno del ángulo entre el vector de la actualización k y el ideal:

$$\cos(\theta^*, \theta^{(k)}) = \frac{(\theta^*)^\top \theta^{(k)}}{\|\theta^*\| \|\theta^{(k)}\|} \geq \frac{k \gamma}{\|\theta^*\| \sqrt{k} R} = \sqrt{k} \frac{\gamma}{\|\theta^*\| R}.$$

El numerador se acotó por abajo con la parte 1 y el denominador por arriba con la parte 2. Como el coseno está acotado por 1:

$$\sqrt{k} \frac{\gamma}{\|\theta^*\| R} \leq 1 \quad \implies \quad k \leq \frac{R^2 \|\theta^*\|^2}{\gamma^2}.$$

Idea clave

Cada update acerca el coseno a 1 en una cantidad finita, o sea acerca el vector actual al ideal. Como el coseno no puede pasar de 1, solo se puede hacer un número finito de updates. Ahí está la convergencia

Ojito...

Está acotado, pero la cota depende del valor de los pesos: puede ser muy grande o muy pequeña, y de antemano no sabemos cuál.

Margen geométrico

El cociente $\|\theta^*\|^2 / \gamma^2$ nos dice qué tan difícil es el problema, o sea cuántos updates voy a tener que hacer. Su inverso es la distancia más chica que hay entre una instancia del conjunto de entrenamiento y mi región de decisión

Si esa distancia es grande hay bastante separación entre la superficie que divide las clases y las instancias, y es más fácil clasificar. Si es chica, es más difícil

Cómo se calcula

Se mide la distancia de la superficie de decisión a uno de los datos de entrenamiento. Sea $\tilde{x}$ el punto de la frontera más cercano a $x^{(t)}$, y ε esa distancia. Moverse de $x^{(t)}$ a la frontera es avanzar ε en la dirección normal, que es la de θ^* :

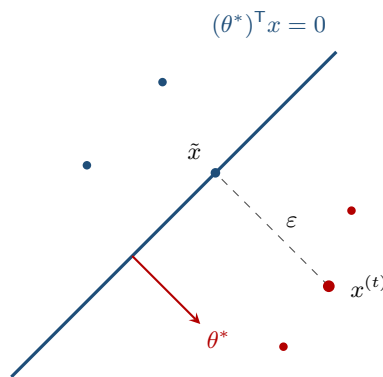
$$\tilde{x} = x^{(t)} - y^{(t)} \varepsilon \frac{\theta^*}{\|\theta^*\|}.$$

Evaluando el clasificador ideal en ese punto:

$$y^{(t)} (\theta^*)^\top \tilde{x} = \underbrace{y^{(t)} (\theta^*)^\top x^{(t)}}_{=\gamma} - \varepsilon \|\theta^*\| = \gamma - \varepsilon \|\theta^*\| = 0,$$

porque $\tilde{x}$ está sobre la frontera y ahí el clasificador vale cero. Despejando queda el **margen geométrico**:

$$\varepsilon = \frac{\gamma}{\|\theta^*\|}.$$



Por lo tanto el k , el número de veces que tengo que actualizar, se puede expresar en términos del margen geométrico:

$$k \leq \frac{R^2}{\varepsilon^2}.$$

Idea clave

Entre más ancho el pasillo que separa las clases, menos updates necesita el perceptrón. La dificultad del problema no está en cuántos datos hay ni en cuántas dimensiones, sino en qué tan apretado quedó ese pasillo

Generalización

Hasta ahorita asumimos que el conjunto de datos era linealmente separable, pero qué pasa si ahora no trabajo con mi conjunto de entrenamiento sino con datos que están fuera de él

Cuando llegue una instancia nueva, el clasificador lineal tendrá que ser capaz de clasificarla: se pone en el plano y se ve si la superficie separadora la separa correctamente

La garantía formal

El argumento se puede hacer preciso. Supongamos que *todas* las imágenes y etiquetas que podríamos llegar a encontrar, no solo las del entrenamiento, cumplen los mismos dos supuestos: $\|x_t\| < R$, y existe un θ^* finito con $y_t(\theta^*)^\top x_t \geq \gamma$ para toda t . O sea que existe un clasificador lineal que sirve para todo el problema, no solo para la muestra.

Ahora imaginemos que las instancias llegan una por una, y que por cada una hacemos a lo más un update si la clasificamos mal, y seguimos adelante sin volver a verla. ¿Cuántos errores vamos a cometer en esa secuencia arbitraria e indefinida?

Idea clave

El mismo número: $k \leq (R/\gamma_{\text{geom}})^2$. La cota no cambia. Una vez cometidos esos errores, todas las instancias nuevas se clasifican correctamente. Eso es una garantía de generalización, y no salió de un supuesto estadístico sobre la distribución de los datos, sino de la geometría del problema.

Ojito...

Con un pero: el algoritmo necesita *saber* cuándo se equivocó, porque la cota está expresada en número de updates, y los updates se hacen sobre errores. En un escenario real donde nadie te dice la etiqueta verdadera, no hay quién dispare el update.

Idea clave

Lo que queremos ahora es un clasificador lineal que nos dé el máximo margen geométrico. De todas las superficies que separan bien el entrenamiento, la que deja el pasillo más ancho es la que tiene más holgura para los datos nuevos. Ese clasificador son las máquinas de vectores de soporte

5. Máquinas de vectores de soporte (SVM)

3 de septiembre de 2026

Resumen rápido. En esta clase vimos como funcionan los SVM

El problema

Se plantea el problema de encontrar la superficie de separación entre los datos de una muestra, de tal manera que el margen geométrico que vimos sea el máximo posible

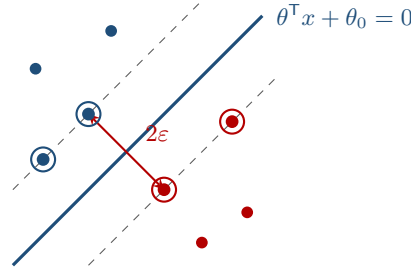


Figura 1: La superficie que deja el pasillo más ancho. Los puntos circulados, los que caen justo sobre el borde del margen, son los vectores de soporte.

Se plantea como un problema de optimización: lo que quiero optimizar son los parámetros de mi vector, de tal manera que exista un margen alrededor de mi superficie de separación que sea lo más grande posible.

Una manera de lograrlo sería encontrar una superficie que separe bien las instancias y luego tratar de que ese clasificador tenga el mayor margen posible. Se busca entonces maximizar el margen geométrico $\gamma / \|\theta\|$, cumpliendo las restricciones

$$y^{(t)}(\theta^T x^{(t)}) \geq \gamma \quad \text{para } t = 1, 2, \dots, n.$$

Idea clave

Esa restricción dice dos cosas al mismo tiempo. El *signo* de $y^{(t)}\theta^T x^{(t)}$ dice si la instancia quedó del lado correcto del plano, y su *magnitud* dice qué tan lejos quedó. Pedir que sea $\geq \gamma$ es pedir las dos: bien clasificada, y además a distancia de sobra, no pegada a la frontera.

Como problema de optimización

Maximizar el cociente que dijimos es lo mismo que minimizar su inverso al cuadrado:

$$\min_{\theta} \frac{1}{2} \frac{\|\theta\|^2}{\gamma^2} \quad \text{sujeto a} \quad y^{(t)}\theta^T x^{(t)} \geq \gamma, \quad t = 1, \dots, n.$$

Así escrito el problema depende de γ , y eso sobra. Si se escala θ y γ por la misma constante, ni las restricciones ni el margen geométrico cambian: la superficie es la misma. Se aprovecha esa libertad para fijar $\gamma = 1$, y el problema queda sin γ :

$$\min_{\theta} \frac{1}{2} \|\theta\|^2 \quad \text{sujeto a} \quad y^{(t)}\theta^T x^{(t)} \geq 1, \quad t = 1, \dots, n.$$

Con esa normalización el margen geométrico es $1/\|\theta\|$, así que minimizar la norma es literalmente ensanchar el pasillo.

El offset

Hasta ahora nuestra función está atada a pasar por el origen, y no queremos eso. Para lograrlo se suma un vector de offset θ_0 :

$$f(x; \theta, \theta_0) = \text{sign}(\theta^\top x + \theta_0),$$

y el problema se vuelve

$$\min_{\theta, \theta_0} \frac{1}{2} \|\theta\|^2 \quad \text{sujeto a} \quad y^{(t)}(\theta^\top x^{(t)} + \theta_0) \geq 1.$$

Esto ayuda a que podamos encontrar una superficie con mayor margen geométrico, porque ya no obligamos al plano a pasar por el origen.

Vectores de soporte

Los elementos que caen justo a la distancia geométrica son los llamados **vectores de soporte**, y son elementos de la muestra. Son los que están circulados en la figura 1.

Propiedades del clasificador de margen máximo

A favor:

- **La solución es única** para cualquier conjunto de entrenamiento linealmente separable. El perceptrón, en cambio, se detiene en cualquier separador que funcione, y cuál depende del orden en que le llegaron los datos.
- **Es robusto a ejemplos ruidosos**, porque dejar la frontera lo más lejos posible de los datos da holgura para que una instancia corrida de lugar siga cayendo del lado correcto.
- **Depende solo de un subconjunto** de los ejemplos, los que caen exactamente sobre el margen. Los demás podrían estar en cualquier parte fuera del margen sin cambiar la solución.

En contra:

Ojito...

Es robusto a ejemplos ruidosos, pero **no a etiquetas ruidosas**. Un solo ejemplo mal etiquetado puede cambiar radicalmente el clasificador de margen máximo: si queda del lado equivocado, el pasillo tiene que encogerse hasta acomodarlo, y la frontera se va a otro lado. Esa fragilidad es la que motiva las variables de holgura.

Validación cruzada

Para evaluar este clasificador podemos hacer uso de la validación cruzada. Consiste en tomar el conjunto de entrenamiento, separar un cachito, construir el modelo con el resto y evaluar en ese cachito. El cachito se llama **fold**, y esto se repite k veces cambiando cuál fold se deja fuera.

Leave-one-out

Si dejamos solo un elemento fuera es el **leave-one-out cross validation**, pero eso se hace n veces, con n el número de elementos del training set. El error se mide así:

$$E_{\text{loo}} = \frac{1}{n} \sum_{i=1}^n \left[1 - \delta(y^{(i)}, f(x^{(i)}; \theta^{(-i)})) \right],$$

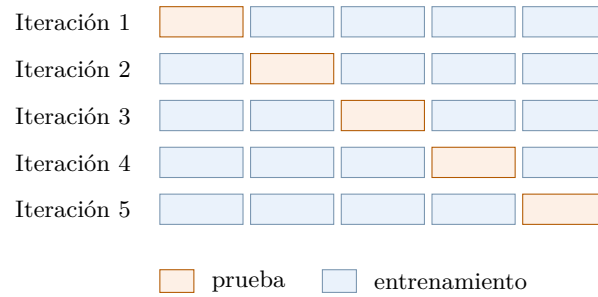


Figura 2: Validación cruzada con $k = 5$. En cada iteración un fold distinto hace de conjunto de prueba.

donde $\theta^{(-i)}$ son los parámetros entrenados con todo el conjunto *menos* el ejemplo i . O sea que cada ejemplo se predice con un modelo que nunca lo vio, y E_{loo} es la fracción de veces que ese modelo se equivoca.

Idea clave

El leave-one-out error está acotado por el número de vectores de soporte:

$$E_{\text{loo}} \leq \frac{\#\text{vectores de soporte}}{n}.$$

Cada vez que se quita una instancia que sea vector de soporte se puede cometer un error, porque quitarla mueve la superficie. Pero quitar una que no lo sea no cambia nada, así que ese ejemplo se sigue clasificando bien. Un clasificador con pocos vectores de soporte generaliza mejor.

Datos con ruido: variables de holgura

Uno de los problemas de este clasificador es que si las etiquetas del conjunto están mal, el clasificador va a estar incorrecto. También hay casos en los que el dato mismo trae error.

Para solucionar esto se introducen **variables de holgura**, dando una optimización relajada:

$$\min_{\theta, \theta_0, \xi} \frac{1}{2} \|\theta\|^2 + C \sum_{t=1}^n \xi_t$$

sujeta a

$$y^{(t)}(\theta^\top x^{(t)} + \theta_0) \geq 1 - \xi_t, \quad \xi_t \geq 0, \quad t = 1, \dots, n.$$

Cada ξ_t mide cuánto se le perdona a la instancia t , y C es un parámetro de costo, o sea qué tanto le va a costar violar las restricciones.

Ojito...

Si el costo no es tan grande, le voy a dar mucho chance a que queden instancias dentro del margen geométrico. Y al revés: con C grande no se permiten violaciones, el modelo se aprieta contra el entrenamiento y vuelve a ser sensible al ruido.

Cuando los datos no son linealmente separables

Todo lo anterior es solo para instancias linealmente separables, pero qué pasa si las instancias no lo son. A alguien se le ocurrió llevar las instancias que no son linealmente separables a un espacio en el que sí lo sean.

Entonces, en lugar de trabajar con la instancia del vector original, se trabaja con su imagen $\varphi(x)$ en el nuevo espacio. Ahí se construye el clasificador lineal y ahí se obtiene la respuesta:

$$y^{(t)}(\theta^\top \varphi(x^{(t)}) + \theta_0) \geq 1 - \xi_t.$$

Idea clave

La superficie sigue siendo un plano, pero en el espacio nuevo. Al regresarla al espacio original se ve curva, y por eso el modelo puede separar datos que una recta no separaba.

6. SVM revisitada: forma dual y kernels

8 de septiembre de 2026

Resumen rápido. Retomamos el problema de mapear las instancias a un espacio donde sí sean separables, y vimos que el mapeo explícito es caro y normalmente ni siquiera lo conocemos. La salida es reescribir la SVM en su forma dual, donde los ejemplos aparecen solo dentro de productos internos, y ahí sustituir el producto por un kernel. Cerramos con cómo se extiende todo esto a más de dos clases.

El mapeo explícito y su costo

La idea de la clase pasada: si el conjunto no es linealmente separable en $\mathbb{R}^d$, se busca una función φ que lleve las instancias a un espacio de mayor dimensión donde sí lo sean,

$$\varphi : \mathbb{R}^d \rightarrow \mathbb{R}^{\mathcal{F}}, \quad \mathcal{F} > d.$$

Ejemplo 6.1. De dos dimensiones a tres, elevando y multiplicando coordenadas:

$$[x_1, x_2] \mapsto \varphi([x_1, x_2]) = [x_1^2, x_2^2, x_1x_2].$$

El producto interno en el espacio nuevo queda

$$\varphi(x)^\top \varphi(z) = x_1^2 z_1^2 + x_2^2 z_2^2 + x_1 x_2 z_1 z_2.$$

Con tres dimensiones de entrada el mismo esquema da nueve coordenadas, todos los productos $x_i x_j$.

Ojito...

Aquí está el problema. Encontrar la φ adecuada no es trivial, y aunque se encuentre, calcular productos internos en $\mathbb{R}^{\mathcal{F}}$ puede salir carísimo: la dimensión crece rápido. Peor aún, en general *no conocemos* φ explícitamente.

La salida no es buscar mejores mapeos, sino reformular la SVM de modo que φ nunca aparezca sola, solo dentro de productos internos. Eso es la forma dual.

La forma dual

El problema en el espacio de características es el de siempre:

$$\min_{\theta, \theta_0} \frac{\|\theta\|^2}{2} \quad \text{sujeto a} \quad y_t(\theta^\top \varphi(x_t) + \theta_0) \geq 1, \quad t = 1, \dots, n.$$

Es convexo y con restricciones lineales, y ese tipo de problemas se puede pasar a su forma dual con multiplicadores de Lagrange. Se introduce un escalar no negativo α_t por cada restricción:

$$J(\theta, \theta_0; \alpha) = \frac{\|\theta\|^2}{2} - \sum_{t=1}^n \alpha_t [y_t(\theta^\top \varphi(x_t) + \theta_0) - 1].$$

Idea clave

Cada α_t es el precio de violar la restricción t . Si la restricción se cumple con holgura, al maximizar sobre α le conviene poner $\alpha_t = 0$: esa restricción no está apretando y no cobra nada. Solo las que se cumplen con igualdad, o sea las que tocan el margen, terminan con $\alpha_t > 0$.

Bajo las condiciones de optimalidad de Lagrange se pueden intercambiar el mínimo y el máximo sin cambiar la respuesta:

$$\underbrace{\min_{\theta, \theta_0} \max_{\alpha \geq 0} J(\theta, \theta_0; \alpha)}_{\text{forma primal}} = \underbrace{\max_{\alpha \geq 0} \min_{\theta, \theta_0} J(\theta, \theta_0; \alpha)}_{\text{forma dual}}.$$

Resolviendo primero el mínimo interno, o sea derivando e igualando a cero:

$$\begin{aligned} \frac{\partial J}{\partial \theta_0} &= - \sum_{t=1}^n \alpha_t y_t = 0, \\ \frac{\partial J}{\partial \theta} &= \theta - \sum_{t=1}^n \alpha_t y_t \varphi(x_t) = 0 \implies \theta = \sum_{t=1}^n \alpha_t y_t \varphi(x_t). \end{aligned}$$

Idea clave

Esa segunda ecuación dice algo fuerte: la solución θ siempre vive en el subespacio generado por los ejemplos de entrenamiento. No es un vector cualquiera de $\mathbb{R}^{\mathcal{F}}$, es una combinación lineal de las propias instancias, pesada por $\alpha_t y_t$.

Sustituyendo esa forma de θ en el objetivo se llega al dual:

$$\max_{\alpha} \sum_{t=1}^n \alpha_t - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j [\varphi(x_i)^{\top} \varphi(x_j)]$$

sujeto a

$$\alpha_t \geq 0, \quad \sum_{t=1}^n \alpha_t y_t = 0.$$

Sigue siendo un problema de programación cuadrática, pero las restricciones son mucho más simples que las del primal.

La matriz de Gram

Fíjate en lo que quedó: la dimensión $\mathcal{F}$ del espacio de características **ya no aparece**. El problema está formulado enteramente sobre la matriz de productos internos, llamada matriz de Gram:

$$K = \begin{bmatrix} \varphi(x_1)^{\top} \varphi(x_1) & \cdots & \varphi(x_1)^{\top} \varphi(x_n) \\ \vdots & \ddots & \vdots \\ \varphi(x_n)^{\top} \varphi(x_1) & \cdots & \varphi(x_n)^{\top} \varphi(x_n) \end{bmatrix}.$$

Es de $n \times n$, o sea que su tamaño depende del número de ejemplos, no de la dimensión del espacio. Ahí está la puerta.

El truco del kernel

Idea clave

El truco del kernel. Si tienes un algoritmo donde los ejemplos aparecen *solo* dentro de productos internos, puedes reemplazar ese producto interno por otro, sin siquiera conocer φ .

Se define entonces una función kernel $K(x, x')$ tal que

$$K(x, x') = \varphi(x)^\top \varphi(x'),$$

y se sustituye. Nunca se calcula $\varphi(x)$: basta con conocer la forma funcional del kernel. Lo que costaba $\mathcal{F}$ operaciones ahora cuesta lo que cueste evaluar K , que suele ser $\mathcal{O}(d)$.

Teorema 6.2 (Mercer). *Para toda función $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ simétrica y semidefinida positiva, existen un espacio de Hilbert $\mathcal{F}$ y una función $\varphi : \mathbb{R}^d \rightarrow \mathcal{F}$ tales que*

$$K(x, x') = \langle \varphi(x), \varphi(x') \rangle.$$

Idea clave

Este teorema es el permiso para trabajar al revés. No hace falta construir el kernel a partir de un conjunto de funciones base $\varphi_1(x), \dots, \varphi_r(x)$: basta definir una K que cumpla las dos condiciones, simetría y semidefinida positiva, y el teorema garantiza que *alguna* φ existe. Nunca hay que verla.

Kernels estándar

Polinomial	$K(x, x') = (1 + x^\top x')^p, \quad p = 1, 2, \dots$
Base radial (RBF)	$K(x, x') = \exp\left(-\frac{\beta}{2} \ x - x'\ ^2\right), \quad \beta > 0$

Con $p = 1$ el polinomial es prácticamente el producto interno original, o sea la SVM lineal de siempre. El RBF corresponde a un espacio de características de dimensión infinita, y aun así se evalúa con una resta, una norma y una exponencial.

Clasificar con la solución dual

Ya con los α_t resueltos, una instancia nueva se clasifica con la función discriminante:

$$\hat{y}(x) = \theta^\top \varphi(x) + \theta_0 = \sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x) + \theta_0,$$

donde SV es el conjunto de vectores de soporte, o sea los que tienen $\alpha_t \neq 0$.

Idea clave

Aquí reaparece la dispersión que ya conocíamos. Muchos α_t salen exactamente cero de la optimización, y son distintos de cero *solo* para los vectores de soporte. Por eso la suma corre sobre SV y no sobre los n ejemplos: los demás no aportan nada al clasificar.

Ojito...

No sabemos qué ejemplos van a ser vectores de soporte hasta haber resuelto la optimización. Y su identidad depende del kernel: cambiar de kernel cambia quiénes son.

De dónde sale θ_0

El offset se cayó del problema dual, así que hay que recuperarlo aparte. Se usa que para todo vector de soporte la restricción se cumple con igualdad:

$$y_i \left(\sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x_i) + \theta_0 \right) = 1, \quad i \in \text{SV},$$

y como $y_i \in \{-1, 1\}$ implica $y_i^2 = 1$, se despeja

$$\theta_0 = y_i - \sum_{t \in \text{SV}} \alpha_t y_t K(x_t, x_i).$$

El margen geométrico en forma kernel

Sigue siendo $1/\|\theta\|$, solo que ahora $\|\theta\|$ se escribe con el kernel:

$$\hat{\gamma}_{\text{geom}} = \left(\sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \right)^{-1/2}.$$

La forma dual relajada

Todo lo anterior supone que los datos sí son separables en el espacio de características. Con variables de holgura el primal era

$$\min \frac{\|\theta\|^2}{2} + C \sum_{t=1}^n \xi_t \quad \text{sujeto a} \quad y_t(\theta^\top \varphi(x_t) + \theta_0) \geq 1 - \xi_t.$$

Su dual es casi idéntico al de arriba. La *única* diferencia es que los multiplicadores quedan acotados por arriba:

$$\max_{\alpha} \sum_{t=1}^n \alpha_t - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad \text{sujeto a} \quad 0 \leq \alpha_t \leq C, \quad \sum_{t=1}^n \alpha_t y_t = 0.$$

Idea clave

Por qué la cota superior: sin ella, una restricción imposible de satisfacer haría crecer su α_t hasta infinito, porque el precio de violarla nunca dejaría de subir. El valor C es justamente el punto donde uno decide dejar de insistir con esa restricción. Es el mismo C del primal, visto desde el otro lado.

Ojito...

Con holgura, θ_0 ya no se despeja con cualquier $\alpha_t > 0$. Hay que usar los que cumplen $0 < \alpha_t < C$, o sea excluir los que están en el tope. Un $\alpha_t = C$ marca un punto que *viola* la restricción de margen, y para ese la igualdad no se cumple.

Valor de α_t	Dónde cae el ejemplo	Sirve para θ_0
$\alpha_t = 0$	Fuera del margen, bien clasificado	No
$0 < \alpha_t < C$	Justo sobre el margen	Sí
$\alpha_t = C$	Viola el margen	No

Clasificación multiclase

Hasta aquí todo fue binario. En multiclase cada punto de entrenamiento pertenece a una de N clases distintas, y se quiere una función que, dado un punto nuevo, determine correctamente a cuál pertenece.

Ojito...

Eso **no** es lo mismo que multitiqueta. Hay escenarios con varias categorías donde un punto puede pertenecer a varias a la vez, y ese es otro problema.

One vs All (OVA)

Se construyen N clasificadores binarios. Para el i -ésimo, los positivos son todos los puntos de la clase i y los negativos todos los que no. Si f_i es ese clasificador, se decide con

$$f(x) = \arg \max_i f_i(x).$$

All vs All (AVA)

Un clasificador por cada par de clases i, j , donde i son los positivos y j los negativos. Como $f_{ji} = -f_{ij}$, en realidad solo hay que entrenar $N(N-1)/2$ de ellos. Se decide sumando votos:

$$f(x) = \arg \max_i \sum_j f_{ij}(x).$$

También se le llama *all-pairs* o *one-vs-one*.

Idea clave

OVA y AVA son tan simples que mucha gente los inventó por separado. Y eligiendo buenos clasificadores binarios de base, funcionan tan bien como cualquier otra cosa que se te ocurra.

	OVA	AVA
Clasificadores	$\mathcal{O}(N)$	$\mathcal{O}(N^2)$
Tamaño de cada uno	Todo el conjunto	Solo dos clases
Balance de clases	Desbalanceado	Balanceado

AVA sale más rápido y usa menos memoria pese a entrenar más clasificadores, porque cada uno es mucho más chico. Si el tiempo de entrenamiento es superlineal en el número de puntos, que es el caso de la SVM, AVA es la mejor opción. OVA además produce muestras de entrenamiento desbalanceadas: una clase contra las otras $N-1$ juntas.

SVM multiclase directa

En lugar de armar varios clasificadores binarios, lo natural es distinguir todas las clases en una sola optimización.

Weston y Watkins (1999). Un vector de pesos w_m por clase, y una holgura $\xi_{i,t}$ por cada ejemplo y cada clase equivocada:

$$\min \quad \frac{1}{2} \sum_{m=1}^k w_m^\top w_m + C \sum_{i=1}^l \sum_{t \neq y_i} \xi_{i,t}$$

sujeto a

$$w_{y_i}^\top \varphi(x_i) + b_{y_i} \geq w_t^\top \varphi(x_i) + b_t + 2 - \xi_{i,t}, \quad \xi_{i,t} \geq 0.$$

Se decide con $\arg \max_m (w_m^\top \varphi(x) + b_m)$. Su desventaja principal es el tiempo de cómputo: el problema cuadrático resultante es enorme.

Crammer y Singer (2001). Misma idea pero con una sola holgura ξ_i por ejemplo, no una por clase:

$$\min \frac{1}{2} \sum_{m=1}^k w_m^\top w_m + C \sum_{i=1}^l \xi_i$$

sujeto a

$$w_{y_i}^\top \varphi(x_i) - w_t^\top \varphi(x_i) \geq 1 - \delta_{y_i, t} - \xi_i,$$

donde $\delta_{i,j}$ es la delta de Kronecker, que vale 1 si $i = j$ y 0 si no. La optimización se hace sin los términos de offset b_m .

SimMSVM (He et al., 2012). Reduce el número de restricciones de margen a l , una por ejemplo, comparando la clase correcta contra el promedio de las demás:

$$w_{y_i}^\top \varphi(x_i) - \frac{1}{k-1} \sum_{m \neq y_i} w_m^\top \varphi(x_i) \geq 1 - \xi_i.$$

Sus autores probaron que su formulación dual es exactamente la de Crammer y Singer con una restricción adicional, y cada modelo de clase se construye sobre sus propios vectores de soporte.

7. Preparación de datos y redes neuronales

10 de septiembre de 2026

Resumen rápido. Antes de aplicar cualquier clasificador hay que preparar los datos: limpiarlos y ponerlos en una escala común, decidiendo entre normalizar y estandarizar. Luego, cómo partir la muestra para entrenar sin caer en overfitting ni en underfitting, y la variante estratificada de la validación cruzada para datos desbalanceados. Al final se abrió el tema nuevo, las redes neuronales artificiales, con la neurona biológica que les sirve de modelo.

Preparación de los datos

A veces queremos aplicar un clasificador, un algoritmo que nos permita separar, clasificar o estimar. Pero antes están los datos, y lo primero es limpiarlos: eliminar columnas vacías, llenar los valores faltantes y demás.

Después viene la escala. Los atributos suelen venir en unidades distintas, y para evitar que unos dominen a otros hay que **normalizar** o **estandarizar**.

Técnica	Qué hace	Fórmula
Estandarizar	Media 0, varianza 1	$z = \frac{x - \mu}{\sigma}$
Normalizar	Todo entre 0 y 1	$x' = \frac{x - \text{mín}}{\text{máx} - \text{mín}}$

Estandarizar reordena los datos en función de su media y su varianza. Eso uniforma el manejo, para que ninguna variable se salga del rango de las demás, y de paso aplasta a los datos atípicos: reduce esa diferencia en vez de dejar que arrastre todo.

Escoger entre las dos es una de las decisiones que hay que tomar, y depende de los datos y del objetivo. La regla que se dio en clase:

Distribución	Qué aplicar
No normal	Estandarizar
Normal	Normalizar

Idea clave

La razón de fondo es de dónde salen las constantes de cada fórmula. Normalizar usa el mínimo y el máximo, que son los dos estadísticos más sensibles que existen a un valor extremo: si la distribución tiene cola larga, un solo punto lejano estira el denominador y comprime a todos los demás en una franja diminuta. Estandarizar usa media y desviación, que no se dejan mover tanto por un punto. Por eso la distribución no normal pide estandarizar. Si la distribución es normal, el mínimo y el máximo se portan bien y normalizar es seguro.

Ojito...

El escalado se aplica **por variable**, no sobre todos los datos juntos. Cada columna tiene su propia media y desviación, o su propio mínimo y máximo. Mezclarlas destruye justo lo que se quería arreglar.

Todo esto sirve para que en el clasificador lineal no haya descompensaciones en los pesos. Si un atributo va de 0 a 1 y otro de 0 a 100 000, el peso del segundo tiene que volverse diminuto para compensar, y el entrenamiento se vuelve lento y mal condicionado.

Cómo dividir el conjunto de muestra

La siguiente decisión es cómo partir la muestra para el proceso de entrenamiento. Los dos extremos fallan por razones opuestas:

- Si le pasamos **todos** los datos al clasificador, se los va a aprender y no va a poder generalizar. Eso es **overfitting**, y además nos quedamos sin manera de medirlo, porque ya no hay datos que el modelo no haya visto.
- Si le pasamos **muy pocos**, no alcanza a aprender el patrón de manera adecuada. Eso es **underfitting**.

De ahí las tres particiones, cada una con su trabajo:

Partición	Para qué
Entrenamiento	Ajustar los parámetros del modelo
Validación	Escoger el modelo y sus hiperparámetros
Prueba	Estimar el error real, una sola vez

Los repartos usuales son 70/15/15 o 80/10/10 por ciento.

Ojito...

El conjunto de prueba se toca una vez, al final. Si se usa para decidir algo, deja de ser una estimación honesta del error: se convierte en parte del entrenamiento, y la generalización que reporta ya está contaminada.

Validación cruzada estratificada

La otra opción es la validación cruzada, que ya se vio en la clase del 3 de septiembre (figura 2): se parte el conjunto en k folds y se rota cuál se deja fuera.

Con datos desbalanceados eso no basta. Se parten **por clases** y los folds se arman con datos de cada clase, de modo que cada fold conserve la proporción original.

Idea clave

Sin estratificar, con una clase rara y folds aleatorios puede tocar un fold sin un solo ejemplo de esa clase. Ese fold no mide nada sobre ella, y el promedio de los k errores sale optimista por una razón que no tiene que ver con el clasificador.

Redes neuronales artificiales

Hasta aquí todos los modelos salieron de plantear un problema de optimización. Pero alguien preguntó por qué no tratamos de emular el modo en que aprende el cerebro humano, y de ahí nace la idea de las redes neuronales artificiales.

La neurona biológica

El cerebro está conectado por neuronas. Las conexiones entre neuronas se llaman **sinapsis**, y se dan entre axones y dendritas.

Parte	Qué hace
Dendritas	Reciben las señales de otras neuronas
Soma	Integra lo que llega y decide si dispara
Axón	Lleva la señal de salida hacia afuera
Sinapsis	Une el axón de una con la dendrita de otra, y es donde la señal se pondera

Por qué es eficiente el cerebro? Por el número: del orden de 10^{11} neuronas y unas 10^{14} conexiones entre ellas. No es que cada neurona sea potente, es que son muchísimas y están densamente conectadas.

La neurona artificial

El modelo artificial se queda con la estructura y tira la biología. Cada parte tiene su contraparte:

Biológica	Artificial
Dendritas	Las entradas $x_1, \dots, x_d$
Sinapsis	Los pesos $w_1, \dots, w_d$
Soma	La suma ponderada más el sesgo
Axón	La salida, pasada por una función de activación

O sea

$$y = g\left(\sum_{i=1}^d w_i x_i + w_0\right),$$

con g la función de activación.

Idea clave

Con g el signo, esa ecuación es exactamente el perceptrón de la clase del 27 de agosto. La neurona artificial no es un modelo nuevo, es el mismo clasificador lineal contado desde la biología. Lo nuevo viene de conectar muchas en capas, que es donde deja de ser lineal.

8. Neurona artificial y separabilidad lineal

15 de septiembre de 2026

Resumen rápido. Continuación de la neurona. El TLU de McCulloch y Pitts con su ejemplo numérico, por qué el escalón es demasiado rígido para modelar una neurona real y cómo lo suaviza la sigmoide, el catálogo de funciones de activación y por qué importa que sean derivables. Al final, la lectura geométrica: el TLU es un clasificador lineal y su frontera es un hiperplano perpendicular al vector de pesos.

El TLU de McCulloch y Pitts

La tarea es modelar algunos de los ingredientes de una red neuronal biológica. El carácter de *todo o nada* del potencial de acción se caracteriza con una señal de dos valores, 0 y 1. Si un nodo recibe n señales de entrada $x_1, \dots, x_n$, cada una solo puede valer 0 o 1.

El efecto modulador de cada sinapsis se encapsula multiplicando la señal que entra por un peso. Con n pesos $w_1, \dots, w_n$ se forman los n productos $w_i x_i$, y cada producto es el análogo de un **potencial postsináptico**, que puede ser positivo o negativo según el signo del peso.

Esos productos se combinan sumándolos, y eso emula lo que pasa en el axón. El resultado es la **activación**:

$$a = \sum_{i=1}^n w_i x_i.$$

Para emular la generación del potencial de acción hace falta un **umbral** θ : si la activación lo alcanza o lo supera, el nodo emite 1; si no, emite 0.

$$y = \begin{cases} 1 & \text{si } a \geq \theta, \\ 0 & \text{si } a < \theta. \end{cases}$$

Definición 8.1. Esa unidad, suma ponderada más umbral con función escalón, se llama **TLU**, *threshold logic unit*, y es de McCulloch y Pitts (1943).

Ejemplo 8.2. Una unidad de cinco entradas con pesos (0.5, 1.0, -1.0, -0.5, 1.2), a la que se le presenta el patrón (1, 1, 1, 0, 0).

La activación es

$$\begin{aligned} a &= (0.5 \times 1) + (1.0 \times 1) + (-1.0 \times 1) + (-0.5 \times 0) + (1.2 \times 0) \\ &= 0.5 + 1.0 - 1.0 + 0 + 0 \\ &= 0.5. \end{aligned}$$

Con umbral $\theta = 0.2$, como $a = 0.5 > 0.2$, la salida es $y = 1$.

Fíjate en el tercer término: el peso $w_3 = -1.0$ es negativo, así que esa entrada **inhibe** en vez de excitar, y se come completo el aporte de la segunda. Los pesos negativos son la contraparte de las sinapsis inhibitorias.

Comunicación con señales no binarias

El escalón es demasiado rígido para lo que hace una neurona real. Se cree que las neuronas codifican su señal en los *patrones* de disparo del potencial de acción, no en la simple presencia o ausencia de un

pulso. Si f es la frecuencia de disparo, está acotada por abajo en cero y por arriba en un $f_{\text{máx}}$ que impone la duración del periodo refractario entre disparos.

La salida es suavizar el escalón hasta una función continua que aplaste el rango, de modo que y dependa de forma suave de a . La forma conveniente es la **sigmoide logística**:

$$y = \sigma(a) = \frac{1}{1 + \exp\left(-\frac{a - \theta}{\rho}\right)}.$$

Desglosando la fórmula pieza por pieza:

- El término $a - \theta$ centra la curva en el umbral: cuando $a = \theta$ el exponente es cero, $\exp(0) = 1$ y queda $y = 1/2$. El umbral es el punto donde la neurona está indecisa.
- El signo menos hace que la curva sea creciente. Si a crece mucho, el exponente se va a $-\infty$, $\exp(-\infty) = 0$ y $y \rightarrow 1$. Si a se hace muy negativa, el exponente se va a $+\infty$ y $y \rightarrow 0$.
- El divisor ρ controla la forma. Valores grandes de ρ achican el exponente y aplanan la curva; valores pequeños la hacen subir más abruptamente. En el límite $\rho \rightarrow 0$ se recupera el escalón. En muchos textos se omite, lo que equivale a fijar $\rho = 1$.

Ojito...

La sigmoide tiende a 1 cuando a crece, pero **nunca lo alcanza**. Igual por abajo: se acerca a 0 sin tocarlo. Nunca satura del todo, y eso va a importar cuando se derive, porque la pendiente nunca es exactamente cero.

Idea clave

La derivada de la sigmoide se escribe en términos de ella misma:

$$\sigma'(a) = \sigma(a) [1 - \sigma(a)].$$

Eso no es una curiosidad, es lo que hace barato el entrenamiento: para calcular la pendiente en un punto no hay que evaluar nada nuevo, se reusa la salida que ya se calculó en el paso hacia adelante. Nota que vale cero en los extremos, donde σ es 0 o 1, y es máxima en $\sigma = 1/2$, o sea justo en el umbral.

Elegir la función de activación

Hay varias opciones, y cada una cambia el comportamiento de la unidad.

Función	Definición	Rango
Escalón	$y = \begin{cases} 1 & a \geq \theta \\ 0 & a < \theta \end{cases}$	$\{0, 1\}$
Sigmoide	$y = \frac{1}{1 + e^{-a}}$	$(0, 1)$
ReLU	$y = \begin{cases} 0 & a < 0 \\ a & \text{si no} \end{cases}$	$[0, \infty)$
Tangente hiperbólica	$y = \frac{e^a - e^{-a}}{e^a + e^{-a}}$	$(-1, 1)$
Softmax	$Y_k = \frac{e^{a_k}}{\sum_i e^{a_i}}$	$(0, 1)$, suman 1

Qué aporta cada una:

- **Escalón.** Decisión dura, sin grises. Es lo que define al TLU original.
- **Sigmoide.** El suavizado del escalón. Su salida en $(0, 1)$ se puede leer como una probabilidad.
- **ReLU.** Deja pasar lo positivo y corta lo negativo. Es barata de calcular y su derivada es 1 o 0, sin aplastarse para valores grandes.
- **Tangente hiperbólica.** Es una sigmoide centrada en cero, con salida en $(-1, 1)$. Que esté centrada ayuda porque las salidas no quedan todas del mismo signo.
- **Softmax.** No opera sobre una neurona sino sobre toda la capa: normaliza las activaciones para que sumen 1. Es la que se usa en la capa de salida de un clasificador multiclase.

Ojito...

El punto que hay que retener: **el escalón no es derivable**. Las otras sí, y eso es esencial para el cálculo eficiente de los pesos. Todo el entrenamiento por descenso de gradiente descansa en poder derivar la función de activación, así que el escalón deja a la unidad sin manera práctica de aprender.

El TLU como clasificador lineal

Un TLU parte sus patrones de entrada en dos categorías según su respuesta binaria. Esas categorías se pueden ver como regiones de un espacio multidimensional separadas por el equivalente en muchas dimensiones de una recta o un plano.

Ejemplo 8.3. Un TLU de dos entradas con $w_1 = 1$, $w_2 = 1$ y $\theta = 1.5$. Las cuatro entradas booleanas posibles dan:

x_1	x_2	Activación	Salida
0	0	0	0
0	1	1	0
1	0	1	0
1	1	2	1

Solo el patrón (1, 1) supera el umbral de 1.5. El TLU está clasificando sus entradas en dos grupos: las que dan 1 y las que dan 0. Como cada patrón tiene dos componentes, se puede dibujar cada uno como un punto del **espacio de patrones**, y ahí (1, 1) queda de un lado de una recta y los otros tres del otro. Esa función es el AND lógico.

De dónde sale la recta

La condición crítica de clasificación ocurre cuando la activación iguala al umbral. Para dos entradas, igualando a y θ :

$$\begin{aligned}w_1x_1 + w_2x_2 &= \theta \\w_2x_2 &= -w_1x_1 + \theta \\x_2 &= -\left(\frac{w_1}{w_2}\right)x_1 + \left(\frac{\theta}{w_2}\right).\end{aligned}$$

Eso es una recta $x_2 = ax_1 + b$, con pendiente $a = -w_1/w_2$ y ordenada al origen $b = \theta/w_2$ sobre el eje x_2 . Con los valores del ejemplo, $w_1 = w_2 = 1$ y $\theta = 1.5$, queda $x_2 = -x_1 + 1.5$: una recta que corta ambos ejes en 1.5 y deja a (1, 1) arriba, solo.

Idea clave

Los pesos fijan la **inclinación** de la frontera y el umbral fija su **posición**. Entrenar es mover esa recta hasta que los puntos queden del lado correcto. Por eso a los TLU se les llama **clasificadores lineales** y a sus patrones, **linealmente separables**. Es el mismo objeto que el perceptrón de la clase del 27 de agosto, visto desde la biología.

En forma vectorial

Usando la definición de producto interno, la activación de un TLU de n entradas se escribe

$$a = W \cdot X,$$

y la condición crítica queda

$$W \cdot X = \theta.$$

Esa ecuación define un **hiperplano perpendicular al vector de pesos**. De un lado quedan todos los patrones que el TLU clasifica como 1; del otro, los que clasifica como 0.

Ojito...

Que el hiperplano sea perpendicular a W no es un detalle decorativo: quiere decir que W apunta hacia el lado de los positivos. Esa observación es la que va a justificar toda la regla de entrenamiento de la clase siguiente, porque corregir un error será *rotar* W hacia el patrón o alejarlo de él.

El umbral como un peso más

Hay un truco de notación que simplifica todo lo que viene. Partiendo de la condición de salida $W \cdot X \geq \theta$, se resta θ de ambos lados y se hace explícito el signo:

$$W \cdot X + (-1)\theta \geq 0.$$

Leído así, el umbral es **un peso más**, alimentado por una entrada fija en -1 . El vector de pesos, que era de dimensión n para una unidad de n entradas, pasa a ser el vector de dimensión $n + 1$

$$[w_1, w_2, \dots, w_n, \theta],$$

que se llama **vector de pesos aumentado**. Con él, la función del nodo se escribe sin umbral a la vista:

$$W \cdot X \geq 0 \implies y = 1, \quad W \cdot X < 0 \implies y = 0.$$

Idea clave

En el espacio aumentado, $W \cdot X = 0$ define el hiperplano de decisión, que sigue siendo ortogonal a W . Y la condición de umbral cero significa que el hiperplano **pasa por el origen**, porque esa es la única forma de que $W \cdot X = 0$ tenga solución en el origen. La ganancia práctica es que ya no hay que entrenar el umbral aparte: entra al mismo saco que los pesos y la misma regla lo ajusta.

9. Descenso de gradiente y backpropagation

17 de septiembre de 2026

Resumen rápido. Cómo se entrena. Primero la regla del perceptrón, que sale de un argumento puramente geométrico sobre rotar el vector de pesos. Luego el descenso de gradiente sobre una función de error, la regla delta con su derivación, y por qué converge donde el perceptrón oscila. Al final el problema del XOR, que obliga a meter capas ocultas, y backpropagation como la respuesta a quién tiene la culpa del error.

Entrenar una red

Ajustar los pesos y los umbrales se hace con un proceso iterativo: se presentan ejemplos de la tarea una y otra vez, y en cada presentación se hacen cambios pequeños para acercarlos a sus valores deseados. A eso se le llama **entrenar la red**, y al conjunto de ejemplos, **conjunto de entrenamiento**.

Trabajando con un solo nodo, el conjunto de entrenamiento son pares $\{V, t\}$, donde V es un vector de entrada y t es la clase objetivo, 1 o 0, a la que V pertenece. La condición de salida 1 se escribe $W \cdot V \geq \theta$.

Ajustar el vector de pesos

Aquí el razonamiento es geométrico y vale la pena seguirlo despacio, porque de él sale la regla.

Caso 1: el objetivo es $t = 1$ pero sale $y = 0$. Para producir un 0 la activación tuvo que ser negativa, así que el producto punto $W \cdot V$ fue negativo. Eso significa que los dos vectores apuntaban en direcciones opuestas, con ángulo mayor a 90° . Lo que hace falta es **rotar W para que apunte más hacia V** , y eso se logra sumándole una fracción de V :

$$W' = W + \alpha V, \quad 0 < \alpha < 1.$$

Caso 2: el objetivo es $t = 0$ pero sale $y = 1$. Ahora la activación fue positiva cuando debió ser negativa, el ángulo es menor a 90° , y hay que **alejar W de V** restando una fracción:

$$W' = W - \alpha V.$$

Las dos situaciones se combinan en una sola regla:

$$W' = W + \alpha (t - y) V$$

Idea clave

El factor $(t - y)$ hace todo el trabajo de selección, y solo puede tomar tres valores. Si acertó, $t - y = 0$ y no se toca nada. Si $t = 1$ y $y = 0$, vale $+1$ y se suma. Si $t = 0$ y $y = 1$, vale -1 y se resta. Una sola expresión cubre los tres casos sin condicionales.

Esa es la **regla de entrenamiento del perceptrón**, de Frank Rosenblatt (1957-1958). El α se llama **tasa de aprendizaje** y controla qué tan grande es cada corrección.

Algoritmo 9.1 Aprendizaje del perceptrón

```

1: repetir
2:   para cada par de entrenamiento  $(V, t)$  hacer
3:     evaluar la salida  $y$  al presentar  $V$  al TLU
4:     si  $y \neq t$  entonces
5:        $W \leftarrow W + \alpha(t - y)V$ 
6:     si no
7:       no hacer nada
8:     fin si
9:   fin para
10: hasta que  $y = t$  para todos los vectores

```

Descenso de gradiente sobre un error

La regla anterior sale de la geometría. Ahora se llega a una parecida desde la optimización, y esa segunda ruta es la que generaliza a redes de varias capas.

Considerando una red de un solo TLU, cualquier función de error E que exprese la discrepancia entre la salida deseada y la real puede verse como una **función de los pesos**:

$$E = E(w_1, w_2, \dots, w_{n+1}).$$

El $n + 1$ es porque el umbral ya va dentro, como peso aumentado.

El vector de pesos óptimo se encuentra minimizando E por descenso de gradiente:

$$\Delta w_i = -\alpha \frac{\partial E}{\partial w_i}.$$

Idea clave

El signo menos es el corazón del método. La derivada $\partial E / \partial w_i$ dice hacia dónde el error **crece** si nuevo ese peso. Como queremos bajarlo, nos movemos en la dirección contraria. Si la pendiente es positiva, el peso baja; si es negativa, sube. Y donde la pendiente es cero, en el mínimo, el peso deja de moverse solo.

Si e^p es el error del patrón de entrenamiento p , el error total es el promedio sobre todos los patrones:

$$E = \frac{1}{N} \sum_{p=1}^N e^p.$$

Cómo definir el error de un patrón

Vale la pena ver los tres intentos, porque cada uno arregla un problema del anterior.

Primer intento: la diferencia. $e^p = t^p - y^p$. No sirve: la combinación $t^p = 0$, $y^p = 1$ da -1 , mientras $t^p = 1$, $y^p = 0$ da $+1$. Los dos casos están igual de equivocados, pero uno cuenta como error negativo y al promediarlos se cancelarían.

Segundo intento: elevar al cuadrado. $e^p = (t^p - y^p)^2$. Ya es simétrico y siempre positivo. Pero hay un problema más fino: y^p depende de a^p a través de la **función escalón, que es discontinua**, así que no se puede derivar.

Tercer intento: medir contra la activación. Como la salida cambia justo cuando la activación cambia de signo, con $a \geq 0$ dando $y = 1$, se define el error respecto a la activación:

$$e^p = (t^p - a^p)^2.$$

Y se agrega un factor de $1/2$, que no cambia dónde está el mínimo pero simplifica las derivadas:

$$e^p = \frac{1}{2} (t^p - a^p)^2.$$

La regla delta

Como E depende de todos los patrones, sus derivadas también, y eso obligaría a presentar el conjunto completo para evaluar un solo gradiente. Se evita presentando un patrón p , evaluando $\partial e^p / \partial w_i$ y usándolo como **estimación** del gradiente verdadero $\partial E / \partial w_i$.

La derivada, paso a paso

Las diapositivas dicen que se puede demostrar. Se demuestra así. Partiendo de $e^p = \frac{1}{2}(t^p - a^p)^2$ con $a^p = \sum_i w_i x_i^p$, y aplicando la regla de la cadena:

$$\begin{aligned} \frac{\partial e^p}{\partial w_i} &= \frac{1}{2} \cdot 2 (t^p - a^p) \cdot \frac{\partial (t^p - a^p)}{\partial w_i} && \text{cadena sobre el cuadrado} \\ &= (t^p - a^p) \cdot \left(-\frac{\partial a^p}{\partial w_i} \right) && t^p \text{ es constante} \\ &= -(t^p - a^p) x_i^p && \frac{\partial}{\partial w_i} \sum_j w_j x_j^p = x_i^p. \end{aligned}$$

Ahí se ve para qué era el $1/2$: se cancela con el 2 que baja del exponente y la expresión queda limpia.

Sustituyendo en $\Delta w_i = -\alpha \partial E / \partial w_i$, los dos signos menos se cancelan y queda la **regla delta**:

$$\Delta w_i = \alpha (t^p - a^p) x_i^p$$

Cómo se lee la fórmula

El gradiente depende de dos cosas, y las dos tienen sentido:

- **De la discrepancia** $(t^p - a^p)$. Entre más grande sea el error, más grande la corrección, y viceversa. Si ya acertó, la corrección es cero.
- **De la entrada** x_i^p . Si esa entrada vale cero, la i -ésima componente no está contribuyendo a la activación, y entonces su peso no tiene culpa de nada: no se toca. Si la entrada es grande, ese peso sí influye mucho y se corrige proporcionalmente.

Algoritmo 9.2 Entrenamiento con la regla delta

- 1: **repetir**
 - 2: **para** cada par de entrenamiento (V, t) **hacer**
 - 3: evaluar la activación a al presentar V al TLU
 - 4: $\Delta w_i \leftarrow \alpha (t^p - a^p) x_i^p$ // para cada peso i
 - 5: $w_i \leftarrow w_i + \Delta w_i$
 - 6: **fin para**
 - 7: **hasta que** la razón de cambio del error sea suficientemente pequeña
-

Esta regla la propusieron Widrow y Hoff (1960), así que originalmente se llamó **regla de Widrow-Hoff**. Se conoce más como **regla delta**, y al término $(t^p - a^p)$ se le dice *la δ* , porque es una diferencia.

Ojito...

La diferencia entre las dos reglas. Si **no existe solución**, o sea si el problema no es linealmente separable, bajo la regla del perceptrón los pesos se siguen alterando en cantidades significativas y el vector **oscila** sin parar. La regla delta, en cambio, siempre converge (con α suficientemente chica) a un vector w_0 donde el error es mínimo, aunque ese mínimo no sea cero y algunos patrones queden mal clasificados. Converger a la mejor respuesta imperfecta es mucho mejor que no converger.

Versión con sigmoide

Se puede reinstalar la salida y^p en la definición del error, ahora que se usa una sigmoide continua en vez del escalón:

$$e^p = \frac{1}{2} (t^p - y^p)^2, \quad y^p = \sigma(a^p).$$

Al derivar aparece un eslabón más en la cadena, la derivada de la sigmoide:

$$\frac{\partial e^p}{\partial w_i} = - (t^p - y^p) \cdot \underbrace{\frac{d\sigma}{da}}_{\sigma'(a)} \cdot x_i^p,$$

y por lo tanto

$$\Delta w_i = \alpha \sigma'(a) (t^p - y^p) x_i^p$$

Idea clave

Ese factor $\sigma'(a)$ gobierna todo. El efecto de cambiar un peso depende de qué tan sensible sea la salida a la activación, y esa sensibilidad es justo la pendiente de la sigmoide. En las colas, donde la sigmoide está casi plana, $\sigma'(a) \approx 0$ y el peso casi no se mueve aunque el error sea grande: la neurona está saturada. El aprendizaje es más rápido cerca del umbral, donde la pendiente es máxima.

El límite: el XOR

Todo lo anterior descansa en que los patrones sean linealmente separables. Hay funciones booleanas elementales que no lo son.

Ejemplo 9.1. Tomando $(0, 0)$ y $(1, 1)$ con $t = 0$, y $(0, 1)$ y $(1, 0)$ con $t = 1$. Los positivos quedan en dos esquinas opuestas del cuadrado y los negativos en las otras dos.

No existe ninguna recta que separe unos de otros: hay que dejar dos esquinas diagonalmente opuestas de un lado, y cualquier recta que deje a $(0, 1)$ y $(1, 0)$ juntas necesariamente encierra también a una de las otras dos.

Esa función es el XOR, y el señalamiento es de Minsky y Papert en *Perceptrons* (1969).

Ojito...

Ese libro frenó la investigación en redes neuronales casi dos décadas. La limitación que señalaba era real, pero era la de *una sola* unidad, no la de la idea entera.

La salida: más capas

La solución se encuentra fácil si el espacio se parte en **tres regiones** en vez de dos. Con dos rectas paralelas se puede dejar a $(0, 1)$ y $(1, 0)$ en la franja de en medio y a las otras dos esquinas afuera, una de cada lado.

1. Si en lugar de una neurona de salida se usan **dos**, se obtienen dos rectas, y con ellas quedan delimitadas las tres regiones.
2. Para escoger entre una zona y otra de las tres hace falta **otra capa**, con una neurona cuyas entradas sean las salidas de las neuronas anteriores.

Eso da una red de tres neuronas repartidas en dos capas: dos neuronas que reciben x_1 y x_2 con pesos $w_{11}, w_{21}, w_{12}, w_{22}$, y una tercera que recibe las salidas de esas dos con pesos w_{31}, w_{32} y produce y .

Definición 9.2. Una capa está **totalmente conectada** cuando las entradas de cada unidad de la capa son las mismas. Una capa toma una entrada $x \in \mathbb{R}^m$ y produce una salida $y \in \mathbb{R}^n$.

Con m neuronas en una capa, el error se suma sobre todas:

$$e^p = \frac{1}{2} \sum_{j=1}^m (t_j^p - y_j^p)^2,$$

y el gradiente del i -ésimo peso de la j -ésima neurona solo involucra el término de error de *esa* neurona:

$$\Delta w_{ji} = \alpha \sigma'(a_j) (t_j^p - y_j^p) x_{ji}^p.$$

Backpropagation

La idea sigue siendo descenso de gradiente sobre E como función de los pesos, pero ahora hay que tomar en cuenta los pesos de dos capas.

Para la **capa de salida** es directo: se tiene acceso al desajuste entre el objetivo y la salida, y la fórmula de arriba aplica tal cual.

El problema son las neuronas de la **capa oculta**, y la pregunta se plantea así: cuánta *culpa* o responsabilidad hay que asignarle a los nodos ocultos por el error que aparece en la salida. Porque claramente, si la capa oculta le está pasando mala información a la de salida, las neuronas de salida no pueden aspirar a acertar.

Descomponer la regla conocida

El punto de partida es mirar la regla que ya se tiene y separarla en piezas:

$$\Delta w_{ji} = \alpha \underbrace{\sigma'(a_j)}_{\text{pendiente}} \underbrace{(t_j^p - y_j^p)}_{\delta^j} \underbrace{x_{ji}^p}_{\text{entrada}}.$$

La entrada y la pendiente de la sigmoide están determinadas únicamente por la **estructura de la neurona**, así que van a aparecer igual en la expresión de una neurona oculta. El término que sobra, $\delta = (t^p - y^p)$, es el específico de las neuronas de salida, y toda la tarea se reduce a encontrar su equivalente para una neurona oculta.

Con eso, la regla de la k -ésima neurona oculta se escribe ya:

$$\Delta w_{ki} = \alpha \sigma'(a_k) \delta^k x_{ki}^p,$$

a falta de saber qué es δ^k .

Encontrar la delta de una neurona oculta

Considera el enlace entre la k -ésima neurona oculta y la j -ésima de salida. Hay dos factores en juego:

- La contribución que la neurona j hace al error está expresada en su propia δ^j , que ya se sabe calcular.
- La influencia que k tiene sobre j está dada por el peso w_{jk} de la conexión entre ellas.

La interacción entre los dos se captura combinándolos **multiplicativamente**, como $\delta^j w_{jk}$. Y como la neurona k casi con seguridad alimenta a varias neuronas de salida, hay que sumar esos productos sobre todas:

$$\delta^k = \sum_{j \in I_k} \delta^j w_{jk}$$

donde I_k es el conjunto de neuronas de la capa de salida que reciben entrada de la neurona oculta k .

Idea clave

Ahí está el nombre del método. La δ de una neurona oculta no se mide contra ningún objetivo, porque no hay objetivo para una capa oculta. Se **arma hacia atrás**, juntando las deltas de las neuronas a las que alimenta, cada una pesada por la fuerza de la conexión. El error viaja de regreso por las mismas aristas que la señal usó para ir hacia adelante, y los pesos hacen de factor de reparto en las dos direcciones.

La regla unificada

Para cualquier neurona k , oculta o de salida, el descenso de gradiente queda en una sola línea:

$$\Delta w_{ki} = \alpha \delta^k x_{ki}^p,$$

y lo único que cambia es cómo se calcula δ^k :

$$\begin{aligned} \delta^k &= \sigma'(a_k) (t_k^p - y_k^p) && \text{si } k \text{ es de salida,} \\ \delta^k &= \sigma'(a_k) \sum_{j \in I_k} \delta^j w_{jk} && \text{si } k \text{ es oculta.} \end{aligned}$$

Algoritmo 9.3 Backpropagation

- 1: presentar el patrón en la capa de entrada
- 2: que las neuronas ocultas evalúen su salida con ese patrón
- 3: que las neuronas de salida evalúen la suya con el resultado del paso anterior
- 4: aplicar el patrón objetivo a la capa de salida
- 5: calcular las δ de las neuronas de salida
- 6: entrenar cada neurona de salida por descenso de gradiente
- 7: para cada neurona oculta, calcular su δ^k
- 8: entrenar cada neurona oculta con la δ^k del paso anterior, por descenso de gradiente

Pasos	Nombre	Qué fluye
1 a 3	Paso hacia adelante	La información, de entrada a salida
4 a 8	Paso hacia atrás	Las δ , de salida a ocultas

El paso 7 es el que propaga las δ de vuelta desde los nodos de salida hacia las unidades ocultas, y de ahí el nombre **backpropagation**. En los años ochenta lo popularizaron Geoffrey Hinton, David Rumelhart y Ronald Williams.

Idea clave

Se puede tener más de una capa oculta, y se entrena exactamente igual: las δ de cada capa, salvo la de salida, se expresan en términos de las de la capa que tienen encima. La recursión es la misma sin importar la profundidad, y eso es lo que vuelve entrenable a una red arbitrariamente honda.

10. Redes convolucionales

22 de septiembre de 2026

Resumen rápido. La primera arquitectura que no conecta todo con todo. Se ve por qué una capa densa no sirve para imágenes, qué hace la convolución, cómo cambia la regla delta cuando un peso se reusa en muchos lugares, y cómo se arma una CNN completa. La segunda mitad es el problema que se destapó al querer apilar capas: el gradiente se desvanece, y ReLU solo lo arregla a medias.

Por qué no basta una capa densa

La clase pasada cerró diciendo que se puede tener cuantas capas ocultas se quiera. Con imágenes eso choca de inmediato con dos problemas.

El primero es el número de pesos. Una imagen de 224×224 en color aplanada da 150 528 entradas. Conectarla a una capa oculta de mil neuronas son 150 millones de pesos, en una sola capa.

El segundo es más de fondo: **aplanar destruye la estructura espacial**. Dos pixeles vecinos quedan como dos coordenadas cualquiera del vector, sin nada que diga que estaban juntos. Y una característica que la red aprende a detectar en una esquina no le sirve de nada en la otra, porque son pesos distintos.

La convolución

En la regla delta de una capa densa, la activación usa la entrada entera:

$$a = \sum_i w_i x_i.$$

En una capa convolucional la neurona no mira toda la imagen de golpe. Mira solo una ventana chica, llamada **filtro** o **kernel**:

$$a_{i,j} = \sum_m \sum_n w_{m,n} \cdot x_{i+m,j+n}.$$

Los índices se leen así:

- (i, j) es la posición de *salida*. Hay una activación por cada lugar donde se pueda apoyar el filtro.
- (m, n) recorre el filtro, no la imagen. Si el filtro es de 3×3 , son nueve términos y ya.
- $x_{i+m,j+n}$ es la ventana de la imagen anclada en (i, j) . El $+i$ y el $+j$ son lo que desliza la ventana.

Ejemplo 10.1. Una imagen de 5×5 con un borde vertical, las dos primeras columnas en 0 y las tres siguientes en 1, y un filtro de 3×3 que detecta bordes verticales:

$$X = \begin{pmatrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}.$$

La activación en la esquina, con la ventana sobre las filas 0 a 2 y las columnas 0 a 2:

$$a_{0,0} = \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} + \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} + \underbrace{(1)(0) + (0)(0) + (-1)(1)}_{-1} = -3.$$

Corriendo la ventana por las nueve posiciones posibles queda un mapa de 3×3 :

$$A = \begin{pmatrix} -3 & -3 & 0 \\ -3 & -3 & 0 \\ -3 & -3 & 0 \end{pmatrix}.$$

La última columna vale cero porque ahí la ventana cae entera dentro de la zona uniforme de unos, y el filtro suma tanto como resta. Las otras dos son distintas de cero porque la ventana cruza el borde. El filtro es ciego a lo plano y solo responde a los cambios.

Canales, paso y relleno

La fórmula esconde tres cosas que en la práctica hay que fijar.

Los canales. Una imagen no es una matriz, es un volumen de alto por ancho por *canales*: tres para color, uno en escala de grises. El filtro tiene la misma profundidad que su entrada, o sea es de $f \times f \times C$, y la suma de la fórmula corre también sobre los canales. Por eso un filtro, por grueso que sea, produce **un solo** mapa de salida de dos dimensiones. Si se quieren K mapas hacen falta K filtros distintos, y esos K mapas son los canales de la capa siguiente.

El paso, o *stride*, es de cuánto en cuánto se desliza la ventana. Con paso 1 se apoya en cada pixel; con paso 2 se salta uno y el mapa de salida sale a la mitad de tamaño.

El relleno, o *padding*, son ceros que se agregan alrededor de la imagen. Sin relleno el mapa encoge en cada capa, y los pixeles del borde participan en menos ventanas que los del centro, así que la red los ve menos.

Juntando los tres, el tamaño de salida de una convolución sobre una entrada de $n \times n$ con filtro de $f \times f$, relleno p y paso s es

$$\left\lfloor \frac{n - f + 2p}{s} \right\rfloor + 1.$$

En el ejemplo de arriba, $n = 5$, $f = 3$, sin relleno y con paso uno, da $5 - 3 + 1 = 3$: por eso la imagen de 5×5 produjo un mapa de 3×3 . Con relleno $p = 1$ habría dado 5, o sea el mismo tamaño que la entrada, que es lo que se hace casi siempre para que la red pueda ser honda sin quedarse sin mapa.

Idea clave

El conteo de pesos de una capa convolucional sale de ahí: $K(f^2C + 1)$, contando un sesgo por filtro. Lo que *no* aparece en esa cuenta es n . El tamaño de la imagen afecta cuánto se tarda la capa, pero no cuántos parámetros tiene, y esa independencia es justo lo que la capa densa no tiene.

Lo que se gana

Tres propiedades, y las tres salen de la misma fórmula:

- **Conexión local.** Cada neurona ve solo su ventana, que se llama su campo receptivo. No hay un peso por pixel de la imagen, hay un peso por celda del filtro.
- **Pesos compartidos.** El *mismo* filtro se desliza por toda la imagen. Un filtro de 3×3 son nueve pesos sin importar si la imagen es de 28×28 o de 1000×1000 .
- **Equivarianza a la traslación.** Si el borde se mueve, la respuesta se mueve con él. Lo que la red aprende a detectar en un lugar le sirve en todos.

Idea clave

Los números del principio con una capa convolucional: 64 filtros de $3 \times 3 \times 3$ son 1 728 pesos, contra los 150 millones de la capa densa. Cuatro órdenes de magnitud menos, y encima conserva la vecindad de los píxeles.

Backpropagation con pesos compartidos

La regla delta de la clase pasada, para una neurona cualquiera:

$$\Delta w_{ki} = \alpha \delta^k x_{ki}^p.$$

Se calcula el error local y se multiplica por la entrada asociada a ese peso. En una convolución hay un detalle: un solo peso $w_{m,n}$ contribuyó a *muchos* píxeles de salida durante el paso hacia adelante, así que recibe culpa desde muchos lugares a la vez:

$$\Delta w_{m,n} = \alpha \sum_i \sum_j \delta_{i,j} \cdot x_{i+m,j+n}.$$

La doble suma recorre exactamente las posiciones donde se aplicó el filtro. No es una regla nueva: es la misma regla delta, acumulada.

Idea clave

Eso no es un truco de redes, es cálculo. Cuando una variable aparece varias veces en una expresión, su derivada es la suma de las derivadas parciales de cada aparición. Compartir el peso obliga a sumar, y por eso la CNN es, palabras del profesor, la aplicación de la regla delta y el descenso de gradiente a un conjunto compacto de pesos reusados sobre toda la superficie de la imagen.

La arquitectura completa

	Etapa	Qué pasa
1	Entrada	La imagen, sin aplanar
2	Extracción de características	Capas convolucionales alternadas con capas de pooling, típicamente max pooling
3	Transición	Aplanado: los mapas finales se estiran en una columna de neuronas de una dimensión
4	Clasificación	Capas densas, y una capa de salida con una neurona por categoría, normalmente con softmax

El **max pooling** lo nombra la diapositiva pero vale la pena decir qué hace: toma el máximo sobre una ventana, casi siempre de 2×2 y avanzando de dos en dos, con lo que cada dimensión espacial se reduce a la mitad. Sirve para dos cosas. Achica el mapa, y da algo de invariancia a traslaciones chicas: si la característica se corre un píxel, el máximo de la ventana normalmente no cambia. No tiene parámetros que entrenar.

Ejemplo 10.2. Un mapa de 4×4 con ventanas de 2×2 y paso dos, o sea cuatro ventanas que no se

traslapan:

$$\begin{pmatrix} 1 & 3 & 2 & 0 \\ 5 & 2 & 1 & 4 \\ 0 & 1 & 7 & 2 \\ 3 & 2 & 1 & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 5 & 4 \\ 3 & 7 \end{pmatrix}.$$

De cada bloque queda solo el máximo. Se conserva *qué tan fuerte* respondió el filtro en esa zona y se tira *en qué pixel exacto* respondió, que es precisamente la información que no importa para reconocer el objeto.

Qué aprende cada capa

Apilar convolución y pooling tiene una consecuencia que da sentido a toda la arquitectura: cada capa ve una región más grande de la imagen original que la anterior, aunque su filtro siga siendo de 3×3 . El campo receptivo crece con la profundidad, y con él crece lo que la capa puede reconocer.

Profundidad	Qué detecta	Sobre qué
Primeras capas	Bordes, esquinas, manchas de color	Unos pocos pixeles
Capas intermedias	Texturas y partes: una rueda, un ojo	Combinaciones de lo anterior
Capas hondas	Objetos completos	Combinaciones de partes

Nadie programa esa jerarquía. Sale sola del entrenamiento, y es la razón por la que en visión la profundidad importa tanto: cada nivel solo puede construirse con las piezas que le pasa el de abajo.

La arquitectura que fijó este esquema es **LeNet**, de Yann LeCun en 1998.

El límite de backpropagation

El problema serio es que **el error se diluye exponencialmente** conforme atraviesa capas de regreso hacia el principio de la red. En una red muy honda solo se entrenan las últimas capas; las primeras casi no se mueven.

La causa está en la regla de la cadena: propagar hacia atrás es multiplicar. De la clase del 15 de septiembre, $\sigma'(a) = \sigma(a)[1 - \sigma(a)]$, que vale a lo más 0.25, en $a = 0$, y se va a cero en cuanto la neurona se satura.

Ejemplo 10.3. Diez capas con sigmoides, todas en su mejor caso:

$$0.25^{10} \approx 9.5 \times 10^{-7}.$$

El gradiente que llega a la primera capa es un millón de veces más chico que el de la última, y eso suponiendo que ninguna neurona esté saturada. Los números de la diapositiva van en la misma dirección: 0.1 en la capa 4, 0.05 en la 3, 0.005 en la 2 y 0.00025 en la 1.

Antes de 2012 todas las redes usaban sigmoide o tangente hiperbólica, y las dos se saturan: con entradas muy positivas o muy negativas la curva se aplanan y su derivada se va a cero. De ahí el nombre, **desvanecimiento del gradiente**.

ReLU

Para resolverlo, los autores de AlexNet metieron ReLU:

$$f(x) = \text{máx}(0, x).$$

Sus dos propiedades, las que según la diapositiva salvaron al aprendizaje profundo:

- **Derivada constante e igual a 1 para $x > 0$.** No encoge nada al multiplicarse en cadena. Para $x < 0$ la derivada es 0 y el error simplemente no pasa.
- **Simplicidad de cómputo.** Una sola operación $\text{máx}(0, x)$, contra la exponencial de la sigmoide.

La referencia es Krizhevsky, Sutskever y Hinton (2012).

Por qué ReLU no alcanzó

Cambiar sigmoide por ReLU permitió que los gradientes fluyeran en redes de 16 o 19 capas, como la VGG. Pero al intentar 50 o 100 capas el gradiente seguía desvaneciéndose. Tres razones:

1. **ReLU moribunda.** Con más capas sube la probabilidad de que alguna neurona reciba solo entradas negativas. Su salida queda en cero, su derivada también, y como el gradiente que la atraviesa es cero, nunca se corrige. La neurona queda muerta.
2. **Gradientes explosivos.** La derivada de ReLU es exactamente 1 del lado positivo, pero el gradiente total en una capa temprana es el producto de los *pesos* de las conexiones por esas derivadas. Si los pesos son mayores que uno, el producto no se desvanece, se dispara.
3. **Degradación del flujo de información.** Apilar demasiadas operaciones no lineales hace que las capas finales pierdan correlación directa con las características básicas de la entrada.

Ojito...

Nótese que 1 y 2 apuntan en direcciones opuestas. El producto de muchos factores casi nunca se queda cerca de uno: o se colapsa o se dispara. Ese es el problema real, y no se arregla escogiendo mejor la función de activación.

Por eso durante años la recomendación fue al revés de lo que hoy se hace: pocas capas ocultas con muchas neuronas, en lugar de muchas capas con pocas neuronas. Así estuvo hasta la aparición del aprendizaje profundo.

11. Redes residuales y autoencoders

24 de septiembre de 2026

Resumen rápido. Las dos respuestas al problema que quedó abierto. Primero ResNet, que le abre al gradiente un camino directo de regreso con una suma. Después los autoencoders, que entrenan sin etiquetas poniendo la entrada como objetivo, y sus dos variantes: el disperso cuando la capa oculta sobra, y el apilado, que arma una red honda entrenando una capa a la vez.

Redes residuales

Una red residual, o **ResNet**, introduce **conexiones de salto**, también llamadas atajos.

La idea es cambiar lo que se le pide aprender a un bloque de capas. En vez de obligarlo a aprender directamente la función $H(x)$, se le pide la **función residual**

$$F(x) = H(x) - x,$$

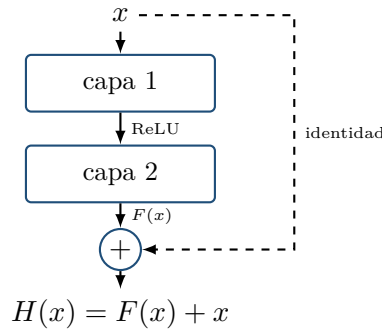
y entonces la salida que de verdad se busca queda

$$H(x) = F(x) + x.$$

Esa suma es la conexión de salto: la entrada x se brinca las capas intermedias y se suma directo a su salida.

Qué hay dentro del bloque

F no es una capa suelta, es un grupo de ellas. En la versión clásica son dos capas convolucionales con una ReLU en medio, y la suma va *antes* de la última ReLU:



Hay un detalle práctico que la fórmula esconde. Para poder sumar, $F(x)$ y x tienen que tener la misma forma, y eso deja de cumplirse en cuanto el bloque cambia el número de canales o reduce el tamaño espacial. Cuando pasa, al atajo se le mete una proyección lineal:

$$H(x) = F(x) + W_s x,$$

que en la práctica es una convolución de 1×1 . Es el único lugar donde el atajo tiene parámetros; en todos los demás es literalmente un cable.

Por qué arregla el gradiente

Al aplicar backpropagation, si la salida es $H(x) = F(x) + x$, la derivada respecto a la entrada es

$$\frac{\partial H(x)}{\partial x} = \frac{\partial F(x)}{\partial x} + 1.$$

Ahí está todo. Siempre hay un término que vale exactamente 1 y que viaja limpio desde la última capa hasta la primera. Aunque las transformaciones del bloque lleven el gradiente a cero, o sea aunque $\partial F/\partial x \approx 0$, el +1 garantiza que lo que llega nunca baje de la señal original.

Comparado con la clase pasada, el contraste es exacto: sin el atajo el gradiente es un *producto* de factores menores que uno, y por eso se colapsa. Con el atajo es una *suma* en la que uno de los sumandos es uno.

Encadenando L bloques, con $x_{l+1} = x_l + F(x_l)$, la regla de la cadena da

$$\frac{\partial E}{\partial x_l} = \frac{\partial E}{\partial x_L} \prod_{i=l}^{L-1} \left(1 + \frac{\partial F(x_i)}{\partial x_i} \right).$$

Al desarrollar el producto, uno de los términos es el puro $\partial E/\partial x_L$, sin multiplicar por nada. Ese sumando sobrevive aunque todos los demás se anulen, y es la ruta directa de la que habla la diapositiva.

Ejemplo 11.1. Cincuenta capas donde cada una atenúa el gradiente a 0.8 de lo que recibe. Sin atajos:

$$0.8^{50} \approx 1.4 \times 10^{-5}.$$

Con atajos, cada factor es $1 + 0.8$ en vez de 0.8, y el término que solo acumula unos vale exactamente 1 sin importar cuántos bloques haya. El gradiente que llega a la primera capa es del mismo tamaño que el que salió de la última.

Aprender la identidad

El otro argumento es igual de importante. Si un bloque no aporta nada al problema, la red puede simplemente ajustar sus pesos para que $F(x) = 0$, y la salida queda $H(x) = x$, la identidad.

Idea clave

Eso explica la observación que originó el método. Una red honda debería ser al menos tan buena como una menos honda, porque siempre podría poner las capas de sobra en identidad. Y sin embargo las redes hondas simples salían *peores*. La conclusión de He y sus coautores fue que el problema no era la capacidad del modelo, era la optimización: aprender la identidad a través de una pila de no linealidades resulta difícil. Con el atajo, la identidad es el caso fácil, el que sale con los pesos en cero.

Antes de ResNet (2015), las redes de más de 20 capas sufrían degradación severa. Con conexiones residuales se volvió posible entrenar de forma estable redes de 152, 500 y hasta más de 1 000 capas. La referencia es He, Zhang, Ren y Sun (2016).

Autoencoders

Un **autoencoder** es un algoritmo de aprendizaje *no supervisado* que aplica backpropagation con un solo cambio: los valores objetivo se ponen iguales a las entradas.

$$t^p = x^p.$$

Con eso el error de la clase del 17 de septiembre se vuelve la distancia entre la entrada y su reconstrucción:

$$e^p = \frac{1}{2} \sum_i (x_i^p - \hat{x}_i^p)^2.$$

No hace falta etiquetar nada. El dato es su propia etiqueta, y por eso cuenta como no supervisado aunque el algoritmo de entrenamiento sea el mismo de siempre.

Las dos mitades, escritas

La red son dos funciones encadenadas. El **codificador** lleva la entrada al código, y el **decodificador** lo trae de vuelta:

$$h = \sigma(Wx + b), \quad \hat{x} = \sigma(W'h + b').$$

A h se le llama el **código** o la representación latente, y a su dimensión el tamaño del cuello. Entrenar es minimizar $\|x - \hat{x}\|^2$ sobre W , W' , b y b' , con la misma backpropagation de siempre.

Una elección común es **amarrar los pesos**, o sea poner $W' = W^T$. Con eso el decodificador deja de tener parámetros propios, la red entrena la mitad de pesos y es más difícil que se memorice el conjunto en lugar de aprender de él.

Ojito...

El nombre de la capa oculta según su tamaño aparece en la literatura y conviene reconocerlo. Si el cuello es *menor* que la entrada, el autoencoder es **incompleto**; si es *mayor*, es **sobrecompleto**. Los dos aprenden, pero por razones distintas, y esa diferencia es justo la que separa esta sección de la que sigue.

La clave está en la capa oculta

Pedirle a una red que copie su entrada suena inútil, y lo sería si no fuera por el cuello de botella. Si la capa oculta tiene *menos* neuronas que las de entrada y salida, la información está obligada a pasar por ese embudo, y la red no tiene más remedio que encontrar una **representación intermedia** que quepa ahí y de la que se pueda reconstruir lo original.

Ya entrenada, la red se parte en dos:

Mitad	De dónde a dónde	Qué hace
Compresor, o codificador	De la entrada a la capa oculta	Produce la versión comprimida
Descompresor, o decodificador	De la capa oculta a la salida	Recupera la versión original a partir de la comprimida

Idea clave

Vale la pena ver con qué se emparenta esto. Un autoencoder *lineal* con un cuello de k neuronas y error cuadrático termina abarcando el mismo subespacio que las primeras k componentes principales, o sea lo mismo que daría una descomposición en valores singulares. Lo que lo vuelve más que eso es la no linealidad de las capas: con sigmoides o ReLU el codificador puede doblar el espacio, y encuentra estructura que una proyección lineal no alcanza.

Para qué se usan

El autoencoder casi nunca se entrena por su salida. Lo que se quiere es alguna de estas cuatro cosas:

- **Reducir la dimensión.** Quedarse con h y tirar el decodificador. Es el uso directo del cuello de botella.
- **Quitar ruido.** Entrenado con entradas sucias y salidas limpias, reconstruye la versión limpia de una entrada que nunca vio.

- **Detectar anomalías.** Aprende a reconstruir bien lo que se parece a sus datos de entrenamiento. Una entrada rara sale con error de reconstrucción alto, y ese error sirve directamente como puntaje de qué tan anómala es.
- **Preentrenar.** Usar los pesos del codificador como punto de arranque de una red supervisada, que es de lo que trata la última parte de la clase.

Autoencoders dispersos

Y si la capa oculta, en vez de tener menos neuronas, tiene *más* que las de entrada y salida?

El riesgo es que la red no aprenda nada: que copie la entrada a un subconjunto de las neuronas ocultas, deje las sobrantes sin usar, y las copie de vuelta a la salida. Reconstruye perfecto y no aprendió absolutamente nada.

La salida es forzar a las neuronas de la capa oculta a activarse *pocas veces*. Entonces la red tiene que volver a aprender un código alternativo, y además disperso. La idea es que los valores de las neuronas ocultas sean cero la mayor parte del tiempo.

Eso limita mucho la expresividad de la capa oculta, pero a cambio obliga a cada neurona a especializarse en cierto patrón de la entrada.

Dos maneras de conseguirlo

- **Autoencoder disperso.** Se agrega al error un factor de dispersión. Si la dispersión es baja, o sea si las neuronas se activan mucho, el factor suma bastante error; si es alta, suma poco. El castigo empuja a la red hacia el código disperso.
- **Autoencoder que quita ruido.** En lugar de dar el mismo valor en la entrada y en la salida, se le mete ruido al vector de entrada y se deja la salida limpia. La red queda obligada a generalizar, porque va a ver varios ejemplos de entrada ligeramente distintos que deben producir la misma salida.

El factor de dispersión no viene escrito en la diapositiva. La forma usual, la de las notas de Andrew Ng que están en las lecturas, mide la activación promedio $\hat{\rho}_j$ de cada neurona oculta contra un objetivo ρ chico, del orden de 0.05, y castiga la diferencia con

$$\beta \sum_j \left[\rho \log \frac{\rho}{\hat{\rho}_j} + (1 - \rho) \log \frac{1 - \rho}{1 - \hat{\rho}_j} \right],$$

que vale cero cuando $\hat{\rho}_j = \rho$ y crece rápido en cuanto la neurona se activa más seguido de lo permitido.

Autoencoders apilados

Un **autoencoder apilado** es una red formada por varias capas de autoencoders dispersos, donde la salida de cada capa se conecta a la entrada de la siguiente. El entrenamiento va por etapas:

1. Entrenar un autoencoder disperso sobre las entradas crudas $x^{(k)}$, para aprender **características primarias** $h^{(1)(k)}$.
2. Usar esas características primarias como si fueran la entrada cruda de *otro* autoencoder disperso, y aprender **características secundarias** $h^{(2)(k)}$, que son más complejas que las primeras.

3. Tratar las secundarias como entrada cruda de un clasificador, y entrenarlo para mapear de ellas a las etiquetas de clase.
4. Juntar las tres piezas en una sola red con dos capas ocultas y una capa final de clasificación.

El método entrena los parámetros de cada capa por separado, mientras mantiene congelados los del resto del modelo. Al terminar esa fase se hace un **ajuste fino** con backpropagation sobre la red completa, para mejorar los resultados moviendo ya todos los parámetros a la vez.

Qué se gana

- **Aprende sin supervisión.** Solo necesita datos, y encuentra por su cuenta las características frecuentes con las cuales etiquetarlos. Las etiquetas hacen falta nada más para el clasificador del final.
- **Se puede ir tan hondo como se quiera.** El problema del error que se diluye no aparece, porque nunca se propaga a través de muchas capas a la vez: cada autoencoder tiene su propio objetivo y es poco profundo.

Idea clave

Las dos mitades de esta clase atacan el mismo problema por lados opuestos. El apilado lo *evita*: nunca propaga a lo largo de toda la red, así que el gradiente no tiene por dónde diluirse. ResNet lo *arregla*: deja que el gradiente recorra la red entera, pero le pone un camino donde no se encoge. Históricamente el preentrenamiento capa por capa fue lo que hizo viable el aprendizaje profundo a mediados de los dos mil. Dejó de usarse casi por completo cuando ReLU, mejores inicializaciones y más datos permitieron entrenar de punta a punta sin esa muleta.

Variante	Capa oculta	Qué la obliga a aprender
Básico	Más chica que la entrada	El cuello de botella: la información no cabe de otro modo
Disperso	Más grande que la entrada	El castigo por activarse seguido
Que quita ruido	De cualquier tamaño	El ruido en la entrada, que impide copiar
Apilado	Varias, una por etapa	Cada capa se entrena sola sobre la salida de la anterior